Manual de seguridad de la base de datos

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías. Motor: **PostgreSQL 14+** · Acceso desde la app: **Sequelize** (`database/conexion.js`).

Este manual distingue entre lo que **ya está implementado** en el proyecto y lo que es una **recomendación** para producción.

1. Qué se protege

Dato	Tabla	Sensibilidad
Correo, nombre y apellido	`users`	Datos personales
Contraseña	`users.password`	Crítica (solo se guarda el hash)
Contenido, likes, seguidores	`posts`, `followers`	Media
Ubicación de imágenes y metadatos faciales	`image_variants`, `face_detections`	Media (edad y género estimados)

Las imágenes **no** viven en la base de datos: solo su ruta o *key* de S3.

2. Estado actual de los controles

Control	Estado	Dónde
Contraseñas con hash bcrypt y sal única por usuario	Implementado	`Controlador/user.js` (`bcrypt.hash(password, 10)`)
Consultas parametrizadas (sin concatenar SQL)	Implementado	Sequelize ORM en todos los modelos
Restricciones de integridad (`UNIQUE`, `CHECK`, `FOREIGN KEY ... ON DELETE CASCADE`)	Implementado	`database/schema.sql`
Credenciales de la BD fuera del código	Implementado	Variables `DB_*` en `.env` (ignorado por git)
Base de datos no expuesta a internet	Implementado	PostgreSQL escucha solo en `localhost` (valor por defecto en Ubuntu) y el Security Group no abre el puerto 5432
Conexión cifrada (SSL)	Opcional	`DB_SSL=true` en `.env`
Usuario de BD con privilegios mínimos	Parcial	El script de despliegue crea un usuario propietario de la BD, no superusuario
Límite de intentos de login	**No implementado**	Ver sección 8
Validación estricta de entradas	**Parcial**	La librería `validator` está instalada pero ningún archivo la usa todavía

3. Control de acceso

Roles. Se usan dos: el superusuario `postgres` (solo administración, por consola local con `sudo -u postgres psql`) y el usuario de la aplicación (`DB_USER`), propietario de la base `mi_red_social`. La aplicación nunca debe conectarse como `postgres` en producción.

Endurecer el usuario de la aplicación:

```
ALTER ROLE app_user NOSUPERUSER NOCREATEDB NOCREATEROLE;
REVOKE CONNECT ON DATABASE mi_red_social FROM PUBLIC;
GRANT CONNECT ON DATABASE mi_red_social TO app_user;
```

Acceso por red. Dejar `listen_addresses = 'localhost'` en `postgresql.conf` y, en `pg_hba.conf`, aceptar solo conexiones locales con contraseña cifrada:

```
host  mi_red_social  app_user  127.0.0.1/32  scram-sha-256
```

Si en el futuro la BD pasa a Amazon RDS, el Security Group de la base debe aceptar el puerto 5432 **únicamente desde el Security Group de la instancia EC2** (ver `SEGMENTACION_RED.md`).

4. Protección de las contraseñas de usuario

- Se guarda únicamente el hash de **bcrypt** (factor de costo 10). El hash incluye una **sal aleatoria única** por usuario, que cumple el requisito del enunciado.
- La contraseña en texto plano nunca se guarda ni se registra en logs.
- Los usuarios que entran con Google reciben una contraseña aleatoria con hash, para que la columna nunca quede vacía.
- Para cambiar el costo en el futuro basta subir el número; los hashes viejos siguen siendo válidos.

5. Protección de la sesión

- Las sesiones usan **JWT** firmado con `JWT_SECRET` (`servicios/jwt.js`) y expiran a los 30 días. El middleware `middlewares/auth.js` rechaza peticiones sin token, con token inválido o vencido.
- `JWT_SECRET` debe ser una cadena aleatoria larga. Si falta, el código usa una clave de desarrollo y, en producción, imprime una advertencia: **nunca desplegar así**.

6. Cifrado

Momento	Medida
En tránsito (navegador a nginx)	HTTPS con Certbot / Let's Encrypt (ver `docs/DESPLIEGUE.md`, paso 6)
En tránsito (app a PostgreSQL)	Mismo servidor: `localhost`. Con RDS: `DB_SSL=true`
En reposo (disco)	Cifrar el volumen EBS al crear la instancia (casilla "Encrypted")
En reposo (S3)	Cifrado del lado del servidor (SSE-S3) activado por defecto en el bucket

7. Gestión de secretos

- `.env` está en `.gitignore`: **nunca** se sube a GitHub.
- Los secretos del proyecto: `DB_PASSWORD`, `JWT_SECRET` y `RECAPTCHA_SECRET_KEY`.
- La aplicación accede a S3 mediante un **rol IAM asociado a la instancia**: no hay claves de AWS guardadas en el servidor ni en el repositorio.
- El usuario IAM del catedrático tiene una clave de acceso con permisos mínimos (iniciar y detener solo esta instancia); se entrega por canal privado y se desactiva después de la revisión.
- Entregar al catedrático los secretos por un canal seguro, no por el repositorio.
- Rotación: si una clave se filtra, cambiarla en `.env`, ejecutar `pm2 restart mi-red-social` y, en el caso de `JWT_SECRET`, tener en cuenta que invalida todas las sesiones abiertas.

8. Brechas conocidas y mejoras recomendadas

Brecha	Riesgo	Mejora
`CORS_ORIGIN=*` por defecto	Cualquier sitio puede llamar a la API	En producción fijar el dominio real, p. ej. `https://miredsocial.com`
Sin límite de intentos de login	Fuerza bruta contra contraseñas	Añadir `express-rate-limit` en `/login/user` y `/registrar/user` (el reCAPTCHA ayuda, pero no sustituye al límite)
Sin cabeceras de seguridad HTTP	Clickjacking, sniffing de tipo MIME	Añadir `helmet` en `index.js`
JWT de 30 días sin revocación	Un token robado sirve hasta vencer	Reducir la vigencia (p. ej. 24 h)
Token guardado en `localStorage`	Lo puede leer un script inyectado	Mantener el escape de texto de React y, si crece el proyecto, pasar a cookie `HttpOnly`
`DB_SSL` con `rejectUnauthorized: false`	Acepta certificados no verificados	Con RDS, usar el certificado raíz de AWS y activar la verificación
`sequelize.sync({ alter: true })` en desarrollo	Cambios de esquema sin control	En producción usar migraciones (ya se desactiva `alter` con `NODE_ENV=production`)

9. Lista de verificación antes de entregar

- [ ] `JWT_SECRET` y `DB_PASSWORD` son valores aleatorios, no los de ejemplo
- [ ] El puerto 5432 **no** aparece en el Security Group
- [ ] `.env` no está en el repositorio (`git ls-files | grep .env` no debe mostrarlo)
- [ ] `CORS_ORIGIN` apunta al dominio real
- [ ] El volumen EBS y el bucket S3 tienen cifrado activado
- [ ] Hay un respaldo reciente y se probó restaurarlo (ver `PLAN_RECUPERACION_DESASTRES.md`)